

IPST-CNAM Albi

Intelligence Artificielle - Code de l'UE : NFP106

Parcours : Master 1 TRIED

Documentation du projet : ChatBot IA Assistant conversationnel intelligent

Domaine choisi : Santé

Auteur : **Mohamed Amine Sobhi**

Professeur : **M. Adrien Escourrou**

Année universitaire : 2025-2026

Août 2026

Table des matières

1	Contexte et besoin	2
2	Fonctionnalités	2
3	Architecture et choix techniques	3
3.1	Bibliothèques : autorisées uniquement	3
3.2	Diagramme UML de classes	4
4	Implémentation des trois piliers	4
4.1	Reconnaissance : NLP Engine	4
4.2	Recherche : KnowledgeBase et SearchEngine	5
4.3	Apprentissage : LearningEngine	7
4.4	Exemple de bout en bout	8
5	Analyse comparative BFS / DFS / A*	9
6	Mécanisme d'apprentissage et boucle de feedback	10
7	Évaluation : les cinq métriques	11
8	Interfaces	12
9	Tests unitaires	12
10	Limites et pistes d'amélioration (M2)	12
11	Usage de l'IA	13
	Conclusion	13

1 Contexte et besoin

Un **chatbot** est un programme capable de répondre en langage naturel aux questions d'un utilisateur. Le besoin traité ici est celui d'un **assistant d'information en santé** : fournir des réponses claires et fiables sur les maladies chroniques, les symptômes, les causes, les traitements et la prévention, à un public non spécialiste souhaitant comprendre un sujet médical avant, éventuellement, d'en parler à un professionnel.

Le projet intègre les **trois piliers fondamentaux de l'intelligence artificielle** étudiés en M1, chacun implémenté *à la main* conformément à l'objectif pédagogique du sujet :

Pilier	Ce qui est implémenté
Recherche	Parcours d'un graphe de connaissances (BFS, DFS, A* avec heuristique admissible) pour trouver la réponse la plus pertinente.
Apprentissage	Amélioration des réponses par TF-IDF, similarité cosinus, Naïve Bayes et boucle de feedback utilisateur.
Reconnaissance	Compréhension de l'intention et des entités de la question (tokenisation, stemming, NER, classification d'intention).

▷ Point clé

Le domaine **Santé** a été retenu car il rejoint mon projet d'alternance, et il se prête naturellement à un **graphe de connaissances** : les concepts médicaux (maladies, symptômes, facteurs de risque, traitements, organes) sont fortement reliés entre eux par des relations typées (*cause, traitement, symptôme, prévention*), ce qui donne tout son sens au pilier « recherche sur graphe ». Les réponses sont fournies à titre informatif et ne remplacent pas un avis médical.

2 Fonctionnalités

- Réponses aux questions sur les maladies, symptômes, causes, facteurs de risque, traitements et prévention (base de **79** paires question/réponse).
- **Classification de l'intention en deux temps** : règles par mots-clés (V1) puis filet de sécurité Naïve Bayes entraîné sur les intentions étiquetées (V2).
- **Extraction d'entités** par correspondance directe (avec normalisation des accents) et distance de Levenshtein (NER approximative).
- **Sélection de la réponse** par un score de graphe (chevauchement d'entités, proximité pondérée, correspondance d'intention), départagé par similarité TF-IDF en cas d'égalité.
- **Apprentissage par feedback** : l'utilisateur note la réponse, ce qui renforce ou pénalise l'association question/réponse et met à jour les poids du graphe.
- **Deux interfaces** : une CLI interactive et une interface web Flask qui *rend visibles les trois piliers* (intention, entités, graphe exploré), avec saisie vocale et lecture à voix haute.

- **Option de reformulation par un LLM (RAG)**, désactivée par défaut : le cœur maison reste autonome, le LLM ne fait que reformuler la réponse trouvée.

3 Architecture et choix techniques

Le système est découpé en **modules indépendants** communiquant par des interfaces Python claires, conformément au sujet. Une séparation stricte est maintenue entre la logique métier (les trois piliers) et les interfaces (CLI, web).

Fichier	Rôle
src/main.py	Orchestrateur ChatBot : pipeline complet <code>answer()</code> .
src/nlp_engine.py	Reconnaissance : tokenisation, stemming, NER, classification d'intention.
src/knowledge_base.py	Graphe de connaissances orienté et pondéré, à relations typées.
src/search_engine.py	Recherche : BFS, DFS, A*, comparaison instrumentée, scoring des réponses.
src/learning_engine.py	Apprentissage : TF-IDF, cosinus, Naïve Bayes, feedback.
src/ui.py / src/web_app.py	Interfaces CLI et web (Flask).
src/config.py	Configuration centralisée + journalisation.
src/llm_engine.py	Reformulation LLM optionnelle (RAG).
src/evaluate.py	Rapport des cinq métriques.

3.1 Bibliothèques : autorisées uniquement

L'objectif étant de **comprendre les algorithmes en les implémentant**, aucune bibliothèque implémentant déjà TF-IDF, Naïve Bayes ou les algorithmes de recherche n'est utilisée. Le tableau ci-dessous résume les choix.

Catégorie	Utilisé
Langage	Python 3.10+
NLP de base	re, unicodedata, collections, json
Structures	heapq, deque, defaultdict
Calcul	math (aucune dépendance à numpy n'a été nécessaire)
Interface web	Flask
Tests	pytest

▷ Point clé

Aucune bibliothèque interdite (sklearn, spaCy, NLTK, tensorflow, pytorch) n'est importée. TF-IDF, la similarité cosinus, le Naïve Bayes multinomial, BFS, DFS, A* et la distance de Levenshtein sont **codés à la main**.

3.2 Diagramme UML de classes

L'orchestrateur ChatBot **compose** les cinq classes des trois piliers (plus le moteur LLM optionnel). La figure 1 présente les classes, leurs principales méthodes et leurs relations de composition.

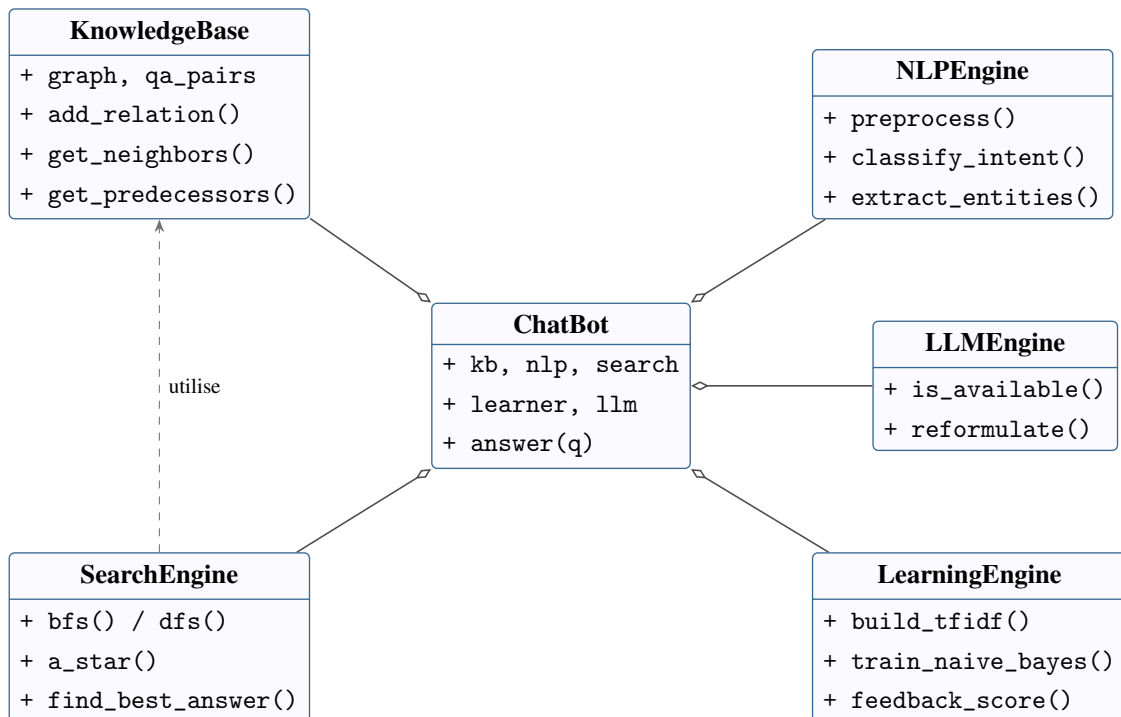


FIGURE 1 – Diagramme UML de classes : ChatBot compose les cinq moteurs (losange de composition); SearchEngine dépend de KnowledgeBase.

4 Implémentation des trois piliers

4.1 Reconnaissance : NLPEngine

La compréhension de la question se fait en trois temps : prétraitement, classification de l'intention, extraction des entités.

Prétraitement. Le texte est mis en minuscules, ses accents et ligatures sont normalisés (obésité → obésite, cœur → coeur), la ponctuation est retirée, les mots vides supprimés, puis chaque token est réduit à sa racine par des règles de suffixes maison.

```
def preprocess(self, text):
    tokens = self.tokenize(text)          # minuscules + strip accents + split
    tokens = self.remove_stopwords(tokens)
    tokens = self.stem(tokens)            # racinisation par regles de suffixes
    return tokens
```

Classification de l'intention (V1 puis V2). On applique d'abord des **règles par mots-clés** (V1); si aucune ne s'applique, on délègue à un classifieur **Naïve Bayes** entraîné au démarrage sur les intentions étiquetées (V2), comme filet de sécurité, exactement le schéma demandé par le sujet.

```
def classify_intent(self, tokens, nb_fallback=None):
    words = set(tokens)
    def has_root(roots):
        return any(t.startswith(r) for t in tokens for r in roots)
    if has_root(['cause', 'origine', 'pourquoi', 'raison']):
        return 'CAUSES'
    if has_root(['sympt', 'signe', 'manifest', 'ressent']):
        return 'SYMPTOMES'
    # ... autres regles (TRAITEMENT, PREVENTION, COMPARAISON, ...)
    if nb_fallback is not None:          # V2 : Naïve Bayes
        predicted = nb_fallback(tokens)
        if predicted:
            return predicted
    return 'DEFINITION'
```

Extraction d'entités (NER). Chaque token est rapproché d'un concept du graphe par correspondance exacte, ou par **distance de Levenshtein** ≤ 1 pour tolérer une petite faute de frappe. La distance est elle aussi implémentée à la main (programmation dynamique).

▷ Point clé

La normalisation des accents *avant* le rapprochement évite d'avoir recours à Levenshtein pour un simple accent : obésité et obesite deviennent identiques, alors que leur distance d'édition brute vaudrait 2.

4.2 Recherche : KnowledgeBase et SearchEngine

Graphe de connaissances. KnowledgeBase représente un **graphe orienté pondéré** (46 concepts, 80 relations). Chaque arête porte, en plus de son poids, un **type sémantique** (cause_de, associe_a, traite_par, symptome, affecte, previent).

```
def get_predecessors(self, concept, types=None):
    """Concepts ayant une relation entrante vers 'concept'.
    Si 'types' est fourni, on ne garde que ces types de relations
    (ou les aretes non typees), afin de ne remonter que les vraies
    causes et d'ignorer les associations bidirectionnelles."""
    result = {}
    for src, weights in self.graph.items():
        if concept not in weights:
            continue
        if types is not None:
            rel = self.relation_types.get((src, concept))
            if rel is not None and rel not in types:
                continue
        result[src] = weights[concept]
    return result
```

► Point clé

Le typage lève l'ambiguïté des **comorbidités bidirectionnelles** (ex : hypertension ↔ AVC, mutuellement facteurs de risque). Sans lui, une question « causes de l'hypertension » remontait à tort la Q/R sur les causes de l'AVC. En marquant ce lien associe_a plutôt que cause_de, les requêtes causales n'utilisent que les vraies causes.

Algorithmes de recherche. SearchEngine implémente BFS (file), DFS (récursif borné) et A* (tas binaire heapq). L'instrumentation compte les nœuds explorés et mesure le temps.

```
def a_star(self, start, goal, heuristic):
    self.nodes_explored = 0
    open_set = [(0, start, [start])]          # (f, noeud, chemin)
    g_scores = {start: 0}
    while open_set:
        f, node, path = heapq.heappop(open_set)
        self.nodes_explored += 1
        if node == goal:
            return path
        for neighbor, weight in self.kb.get_neighbors(node).items():
            g = g_scores.get(node, float('inf')) + (1.0 - weight)
            if g < g_scores.get(neighbor, float('inf')):
                g_scores[neighbor] = g
                h = heuristic(neighbor, goal)
                heapq.heappush(open_set, (g + h, neighbor, path + [neighbor]))
    return None
```

Heuristique admissible. L'heuristique utilisée est $h(n) = 1 - \text{Jaccard}(\text{bigrammes}(n), \text{bigrammes}(\text{goal}))$. Elle est **admissible** : la similarité de Jaccard est toujours ≤ 1 , donc $h(n) \geq 0$, et comme le coût réel d'une arête ($1 - \text{poids}$) est lui aussi ≥ 0 , h ne peut jamais surestimer le coût restant. Deux concepts orthographiquement proches partagent davantage de bigrammes, ce qui guide la recherche sans jamais la biaiser vers un chemin sous-optimal.

Sélection de la réponse. `find_best_answer` note chaque Q/R candidate :

$$\text{score} = 2|O| \cdot \frac{|O|}{|C|} + \sum_{c \in C} \text{poids}(e, c) + 10 \cdot [\text{intention}]$$

où O est le chevauchement entre entités et concepts C de la Q/R. La prime de +10 en cas de correspondance d'intention est volontairement dominante, ce qui garantit qu'une question de type *traitement* ne renvoie pas la *définition* du même concept.

4.3 Apprentissage : LearningEngine

TF-IDF et similarité cosinus (maison). La matrice TF-IDF et la similarité cosinus sont calculées sans bibliothèque externe.

```
def cosine_similarity(self, vec_a, vec_b):
    common = set(vec_a) & set(vec_b)
    dot = sum(vec_a[t] * vec_b[t] for t in common)
    norm_a = math.sqrt(sum(v * v for v in vec_a.values()))
    norm_b = math.sqrt(sum(v * v for v in vec_b.values()))
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return dot / (norm_a * norm_b)
```

Naïve Bayes multinomial (maison). L'entraînement estime les probabilités a priori des classes et les vraisemblances des mots avec **lissage de Laplace** ; la prédiction se fait en **log-probabilités** pour éviter les débordements numériques.

```
def predict_intent(self, tokens):
    best_class, best_score = None, float('-inf')
    for c in self._nb_classes:
        score = math.log(self._nb_prior.get(c, 1e-10))
        for token in tokens:
            score += math.log(self._nb_likelihood.get(c, {}).get(token, 1e-10))
        if score > best_score:
            best_score, best_class = score, c
    return best_class
```

Le mécanisme de feedback est détaillé à la section 6.

4.4 Exemple de bout en bout

Pour montrer comment les trois piliers coopèrent sur une même requête, on suit la question « *Quelles sont les causes de l'hypertension ?* » de l'entrée jusqu'à la réponse. Le tableau 1 donne l'état du système après chaque étape.

Étape	Pilier	Résultat
Prétraitement	Reconnaissance	[quel, caus, hypertension] (minuscules, accents normalisés, mots vides retirés, racinisation).
Classification	Reconnaissance	La règle <code>has_root(['cause', ...])</code> déclenche l'intention CAUSES.
Extraction (NER)	Reconnaissance	hypertension est rapproché du concept homonyme du graphe (correspondance exacte).
Recherche des causes	Recherche	<code>get_predecessors('hypertension', {cause_de})</code> remonte les arcs <i>typés</i> : obésité, tabagisme, stress.
Sélection de la réponse	Recherche	<code>find_best_answer</code> note chaque Q/R : la prime d'intention (+10) fait gagner la paire « <i>causes de l'hypertension</i> ».
Ajustement	Apprentissage	<code>feedback_score</code> vaut 0 sans historique ; le classement reste celui du graphe.

TABLE 1 – Trace complète d'une requête à travers les trois piliers.

▷ Point clé

Le **typage des relations** est décisif à l'étape de recherche : sans lui, `get_predecessors` remonterait aussi les arcs *associe_a* ou *symptôme*, et la question *causale* renverrait des *symptômes* de l'hypertension au lieu de ses *causes*. Le filtre `{cause_de}` garantit une réponse sémantiquement correcte.

Si l'utilisateur note ensuite cette réponse ≥ 4 , un `feedback_score` positif viendra la renforcer lors des requêtes similaires ; s'il la note ≤ 2 , la paire est pénalisée et une autre réponse

candidate peut passer en tête : c'est le mécanisme détaillé à la section 6.

5 Analyse comparative BFS / DFS / A*

`SearchEngine.compare_algorithms` exécute les trois algorithmes sur une même requête et relève, pour chacun, le chemin trouvé, le nombre de nœuds explorés et le temps. La comparaison porte sur **12 requêtes** du graphe de connaissances santé (`benchmark_search.py`).

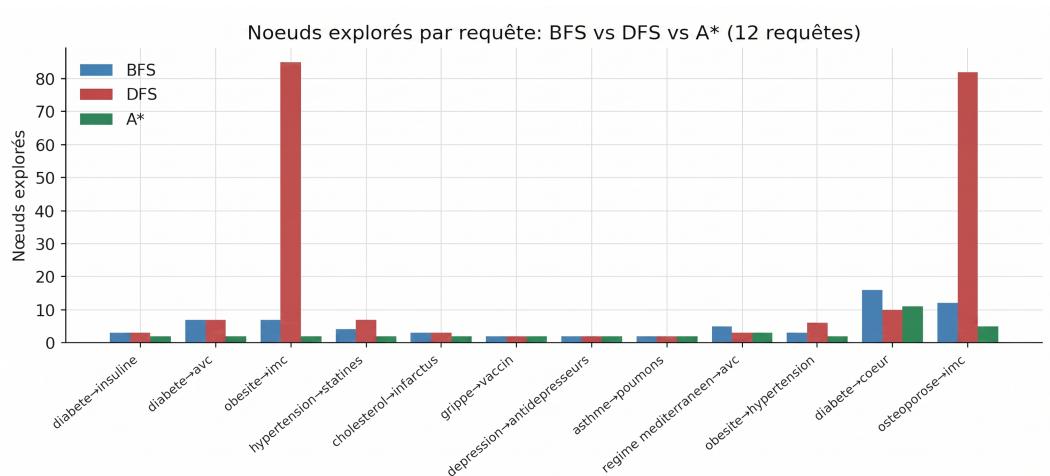
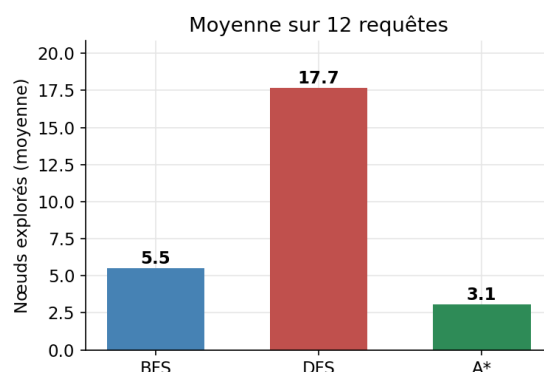


FIGURE 2 – Nœuds explorés par requête pour BFS, DFS et A*. A* explore systématiquement le moins de nœuds ; DFS explose sur les requêtes où la première branche suivie est mauvaise (*obesite*→*imc* : 85 nœuds).

Algorithme	Nœuds explorés (moy.)	Garantie	Guidage
BFS	5,5	chemin le plus court	aucun
DFS	17,7	non optimal	aucun
A*	3,1	chemin optimal (h admissible)	heuristique



Lecture. A* explore en moyenne **3,1** nœuds contre **5,5** pour BFS : l'heuristique concentre la recherche vers le but. DFS, sans guidage et sans garantie d'optimalité, visite en moyenne **17,7** nœuds et peut suivre de longues branches inutiles avant de trouver le but.

▷ Point clé

Double enseignement : BFS garantit le chemin le plus court mais explore « en largeur » sans direction ; A*, grâce à une heuristique admissible, conserve la garantie d'optimalité *tout en* réduisant le nombre de nœuds explorés. C'est exactement le comportement attendu ($A^* < BFS$).

6 Mécanisme d'apprentissage et boucle de feedback

Conformément au sujet, le feedback utilisateur **modifie effectivement** le comportement du système et **améliore les réponses futures**. Deux effets se combinent :

1. **Ajustement direct du classement** (feedback_score) : une note ≥ 4 *renforce* l'association question/réponse, une note ≤ 2 la *pénalise*. Cet ajustement est ajouté au score de graphe au moment de la sélection. Sans aucun feedback, il vaut zéro : le comportement par défaut est strictement inchangé.
2. **Ré-entraînement** (retrain) : tous les trois retours, le TF-IDF est reconstruit sur les réponses bien notées et les poids du graphe sont ajustés ($\pm 0,05$).

```
def feedback_score(self, question, candidate_answer):
    if not self.feedback_log:
        return 0.0 # neutre sans feedback
    q_tokens = set(self._tokenize_candidate(question))
    total = 0.0
    for entry in self.feedback_log:
        if entry.get('answer') != candidate_answer:
            continue
        e_tokens = set(self._tokenize_candidate(entry.get('question', '')))
        jac = len(q_tokens & e_tokens) / len(q_tokens | e_tokens)
        if jac >= 0.8: # question quasi identique
            total += entry.get('score', 3) - 3
    return total * self.FEEDBACK_GAIN
```

Démonstration sur 3 cycles. Les 79 Q/R étant déjà répondues à 100 %, la progression se mesure sur un **jeu de reformulations inédites** (questions « utilisateur » jamais vues, ex. : « À quoi sert l'insuline ? »). Certaines sont initialement mal routées ; en trois cycles où l'utilisateur pénalise la mauvaise réponse et enseigne la bonne, la précision passe de **73,3 %** à **93,3 %**.

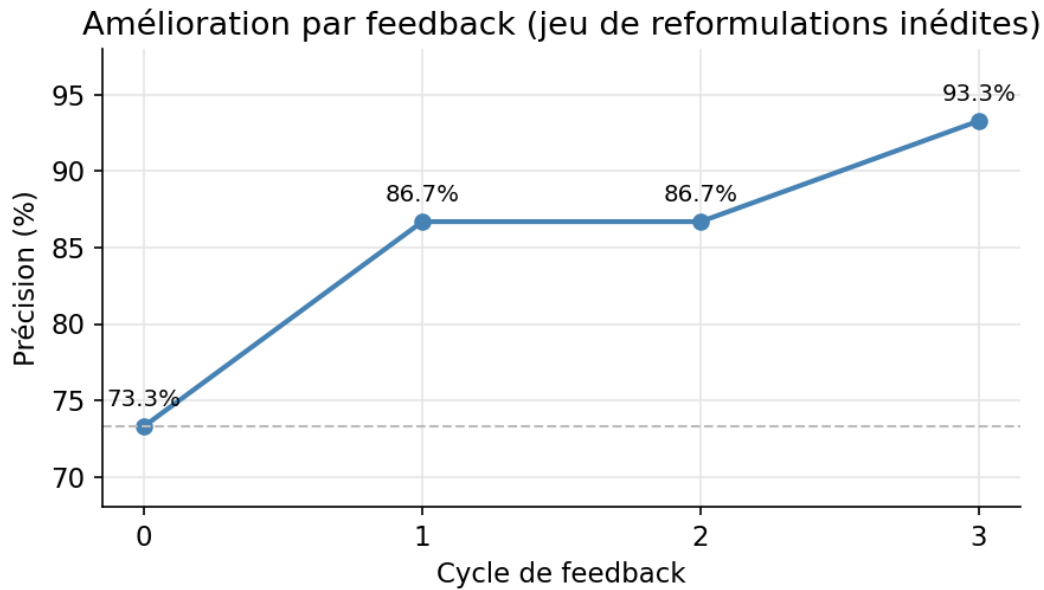


FIGURE 3 – Précision sur le jeu de reformulations au fil des trois cycles de feedback : 73,3 % → 93,3 % (+20 points).

▷ Point clé

La 15^e question (« Pourquoi devient-on obèse ? ») reste non corrigée : le mot « obèse » n'est pas reconnu comme le concept *obesite* (distance de Levenshtein 2), donc aucune entité n'est extraite et aucune réponse candidate n'existe : le feedback est alors hors de portée. Cette limite est assumée plutôt que masquée.

7 Évaluation : les cinq métriques

Le script `evaluate.py` calcule les cinq métriques demandées. Les valeurs ci-dessous sont **mesurées** (non estimées) sur la base actuelle.

Métrique	Résultat mesuré	Cible
Précision des réponses	100 % (79/79)	$\geq 70\%$
Précision des intentions	100 % (79/79)	$\geq 80\%$
Temps de réponse moyen	≈ 1 ms	≤ 2 s
Nœuds explorés (12 requêtes)	BFS 5,5 vs A* 3,1	$A^* < BFS$
Gain par feedback (3 cycles)	+20 pts (73,3% → 93,3%)	$\geq +5\%$

▷ Point clé

Les cinq cibles du sujet sont atteintes. La précision des réponses est mesurée par comparaison exacte à la réponse de référence ; la précision des intentions compare l'intention prédite à l'intention étiquetée ; le gain de feedback est mesuré sur le jeu de reformulations décrit ci-dessus.

8 Interfaces

CLI. `python src/main.py` lance une boucle question/réponse avec demande de note (1–5) et ré-entraînement automatique tous les trois retours.

Web (Flask). `python src/web_app.py` sert une page de discussion. Au-delà de la réponse, l'interface **rend visibles les trois piliers** : un panneau « Analyse » affiche l'intention détectée et les entités extraites (reconnaissance), et un **graphe de connaissances interactif** dessine le sous-graphe exploré autour des concepts, arêtes colorées par type de relation et nœuds cliquables (recherche). La page propose aussi la saisie vocale, la lecture à voix haute et un interrupteur activant la reformulation par LLM.

9 Tests unitaires

57 tests pytest au cœur (et jusqu'à 85 avec les interfaces et options), répartis par module et tous au vert. Ils vérifient notamment :

- KnowledgeBase : `add_concept`, `add_relation`, `get_neighbors`, `get_predecessors` (dont le filtrage par type);
- SearchEngine : BFS/DFS/A* trouvent un chemin, heuristique admissible, désambiguïsation des relations bidirectionnelles;
- LearningEngine : TF-IDF, cosinus, Naïve Bayes, et le renforcement / pénalisation par feedback;
- ChatBot : pipeline complet, correction d'un mauvais routage par le feedback, comportement inchangé lorsque le LLM est désactivé.

```
pip install -r requirements.txt
pytest tests/                # tests unitaires
python src/evaluate.py      # les 5 metriques
python src/main.py         # interface CLI
python src/web_app.py       # interface web (http://127.0.0.1:5000)
```

10 Limites et pistes d'amélioration (M2)

- **NER par règles** : le stemming par suffixes et la distance de Levenshtein atteignent leurs limites (ex. « obèse » $\neq$ obesite). Une table de synonymes, puis des **embeddings denses**, élargiraient la couverture.
- **Heuristique A*** basée sur des bigrammes de caractères : proxy simple, sans vraie sémantique. En M2, une heuristique fondée sur la similarité cosinus entre embeddings denses

serait plus fine.

- **Typage manuel des relations** : à maintenir à jour lors de l'ajout de nouvelles relations ; une inférence automatique du type est envisageable.
- **Pas de contexte conversationnel** multi-tour ; **base limitée** à 79 Q/R.
- Pistes M2 annoncées par le sujet : deep learning (transformers, génération), reinforcement learning, architecture distribuée pour la base de connaissances.

11 Usage de l'IA

Conformément aux règles concernant l'usage de l'IA, je précise ici où j'ai utilisé un assistant IA (Claude, Anthropic) et où je ne l'ai pas fait.

Le cœur du projet, je l'ai écrit moi-même. Les trois piliers et leurs algorithmes sont de ma main : tokenisation, stemming et NER par distance de Levenshtein, classification d'intention, graphe de connaissances, parcours BFS/DFS/A* avec heuristique admissible, TF-IDF, similarité cosinus, Naïve Bayes et boucle de feedback. Cela couvre les fichiers `nlp_engine.py`, `knowledge_base.py`, `search_engine.py`, `learning_engine.py` et `main.py`.

L'IA m'a servi sur le pourtour du projet, sans toucher à la logique des piliers :

- l'interface web Flask (`web_app.py`, `templates/`, `static/`);
- la suite de tests `pytest` (dossier `tests/`);
- la configuration et la journalisation centralisées (`config.py`);
- le connecteur LLM optionnel de reformulation (`llm_engine.py`);
- l'extension des données JSON (`qa_pairs`, `knowledge_graph`) et le script de mesure des métriques (`evaluate.py`);
- du débogage (scoring, normalisation des accents), et la rédaction brève du README et de cette documentation.

Les portions de code écrites avec l'assistance de l'IA sont encadrées dans les fichiers par un bandeau # `CODE IA (Claude - Anthropic)`. Chaque proposition a été relue, testée puis intégrée à la main ; la validation s'appuie sur l'exécution des tests unitaires et sur les cinq métriques mesurées par `evaluate.py`.

Conclusion

Ce projet m'a fait mettre en œuvre, de bout en bout et *sans bibliothèque toute faite*, les trois piliers de l'IA sur un cas concret : un assistant santé fondé sur un graphe de connaissances. La partie la plus formatrice a été de constater comment ces piliers **se combinent** : la reconnaissance transforme la question en intention et entités, la recherche exploite le graphe typé pour trouver la meilleure réponse, et l'apprentissage ajuste ce choix au fil des retours.

La comparaison BFS/DFS/A* illustre un point théorique marquant : une heuristique **admissible** permet à A* de réduire le nombre de nœuds explorés *tout en* conservant la garantie d'optimalité. Le typage des relations, introduit pour lever l'ambiguïté des comorbidités, montre par ailleurs qu'un graphe de connaissances gagne à porter de la **sémantique** et pas seulement des poids.

Les limites identifiées (NER par règles, heuristique syntaxique, absence de contexte conversationnel) ouvrent directement sur le M2 : embeddings denses, modèles de deep learning et architecture distribuée.